Кейс → решение → доказательство

Этот документ отвечает на один вопрос: **что требовалось от решения, что мы для этого сделали и чем это можно проверить своими руками**. Формулировки требований кейса и ответов организаторов пересказаны. Для каждого пункта дано четыре вещи: проблема простыми словами, наше решение, **конкретный пример** (экран, запрос с ответом, строка конфигурации или шаг демо-сценария) и доказательство (файл теста, число из валидации, результат пилота).

Как повторить примеры. Поднимите стенд (README, DEPLOY.md), выполните `make demo`, откройте `/jury` — регистрация не нужна: там одноразовый кандидат, вход в демо-компанию «Северный код» и в лабораторию. Логины демо-аккаунтов: `make creds`. Все примеры ниже проверены 9 октября 2026 на этом коде в режиме `VZ_DEMO_MODE=true` на демо-мире (около 300 синтетических кандидатов). Номера вида «К-2118» — псевдонимы: они зависят от секрета стенда, у вас будут другие, а структура ответа та же.

Статусы. «выполнено» — работает и проверено тестами; «частично» — работает, но есть оговорка, она написана тут же; «не сделано» — честно сказано. Сводка пробелов — в разделе 11.

Что такое валидация в этом документе. Все числа «качества» — симуляция на синтетической популяции с известной истиной (VALIDATION.md) плюс пилот с ИИ-агентами в роли кандидатов. Живых людей в проверках не было, и мы это нигде не скрываем.

Содержание: 0. Кейс на одном экране · 1. Обратный подбор и контакт · 2. Категория и тест · 3. Кабинет кандидата · 4. Кабинет работодателя · 5. ФСП · 6. Зарплата, правила видимости, безопасность · 7. Валидация на ваших данных · 8. Документы, API, запуск · 9. Ответы организаторов · 10. Сверх кейса · 11. Чего нет

0. Кейс на одном экране

Проблема кейса	Наш ответ	Главное доказательство	Подробно
Резюме не сопоставимы и им нельзя верить	Уровень задаёт адаптивный тест, а не текст; резюме даёт только серые чипы «заявлено»	Точный грейд по стажу — 0,54, по тесту — 0,75; накрутка навыков в самоописании сдвигает нас на 0,0 позиций, поиск по резюме — на 518	2.1, 1.6
Единый набор заданий утекает, «ИИ под каждого» ненадёжен	Семейства заданий и одноразовые клоны, ключ считается исполнением кода	Утечка 20 кандидатов: ложный Middle 0,10 против 1,00 у фиксированной формы	2.2
Задания решают чат-боты	«Двойник в буфере» подменяет скопированное условие; живая сверка после «да»	Пилот с ИИ-агентами: «списывающие» не получили ложный Middle ни в одном из 4 прогонов (выборка мала)	2.3, 10.1
Работодатель тонет в резюме	Описание потребности текстом с обязательной вилкой → слепая объяснимая подборка	P@10 0,61 против 0,13 у поиска по резюме на тех же брифах	1.1, 1.2
Непонятно, почему именно этот кандидат	Для каждого: три довода «за» и один «против», каждый со ссылкой на доказательство; сумма вкладов равна Score	Σ вкладов = Score в 100 % карточек; довод ссылается на существующее доказательство в 100 %	1.2
Сценарий контакта: кто, когда, на каких условиях	Приглашение с вилкой → «Да / Да, если… / Нет» → контакты только после взаимного «да» и отдельного согласия	`test_contact_privacy.py`, `test_api_contact_lifecycle.py`; в слепой карточке нет ни одного ПДн-ключа	1.3, 1.4
Участие в ФСП не должно быть входным билетом	Достижения ФСП двигают ранг внутри категории; нет истории — F = 0, это никогда не «против»	«Отсутствие ФСП не в “против”» 1,00; экспозиция без ФСП 0,91 при допуске 0,80–1,25	5.2

Проблема кейса	Наш ответ	Главное доказательство	Подробно
Нужна честная проверка на своих данных	Наш набор пар, свой набор, своя вакансия — на странице `/lab/check` и в API; 100 брифов, протокол записан до прогона	38 из 43 порогов выполнены, 5 нет — перечислены с причинами	7
Сдача: документы, API, запуск, нагрузка	19 документов, OpenAPI на 242 операции, Docker Compose, нагрузочный тест	100 пользователей: 24 739 запросов, 0 сбоев, p95 170 мс	8

1. Обратный подбор и контакт

Это ядро кейса: инициатор — работодатель, но решение принимают обе стороны.

1.1. Работодатель описывает потребность текстом, а не листает резюме — выполнено

- **Проблема.** Классический найм — это поток резюме. Нужно, чтобы работодатель описал, кто нужен, а система нашла людей сама, и чтобы без зарплатной вилки ничего не отправлялось.
- **Решение.** Экран `/employer/needs/new` : свободный текст, живая карточка разбора с цитатами из текста, обязательная вилка, необязательный блок «о команде». Разбор — правила и словари; LLM по умолчанию выключен и в ранжировании не участвует.
- **Пример (запрос и ответ).** `POST /api/needs/parse` с текстом «Бэкенд в команду платёжного шлюза. Python, FastAPI, PostgreSQL, опыт от 2 лет. Kafka и Docker будут плюсом. Гибрид, Москва. 220–280 тыс. на руки» возвращает:
  - вилка 220 000–280 000, «на руки», с цитатами «220–280 тыс.» и «на руки»;
  - Python, FastAPI, PostgreSQL — `must` ; Kafka и Docker — `nice` с причиной `marker` и цитатой «будут плюсом»;
  - специализация `backend_python` с вероятностью 0,93; грейд Middle 0,69, Senior 0,31 (то есть система сама показывает, что не уверена между ними);
  - `"llm_used": false` .

  Тот же текст без вилки: `salary_check.status = "missing"` , сообщение «Укажите вилку: от и до, в рублях в месяц», а `POST /api/needs` отвечает `422 salary_missing` .
- **Доказательство.** `tests/test_matching_brief.py` , `tests/test_api_needs_rules.py` ; разбор брифа на 20 test-брифах: специализация top-1 0,90, грейд точно 0,95, F1 навыков 0,98 (VALIDATION.md п. 4.3); на 60 новых брифах: специализация 0,97, грейд 0,97 (п. 4.1а). Разбор написан правилами, поэтому детерминирован.

1.2. Слепая объяснимая подборка — выполнено

- **Проблема.** «Чёрный ящик» ранжирования нельзя ни проверить, ни оспорить; кроме того, ФИО и бренды прежних работодателей запускают оценку по ярлыку.
- **Решение.** `Score = 100 · G · Cov · Cond · Σ(w·x)` с пятью компонентами: тест T, стек K, ФСП F, свежесть A, условия U (веса в `config/scoring.yaml` : 0,40 / 0,25 / 0,15 / 0,10 / 0,10). Карточка слепая: категория, полоса теста, модули ✓/✓✓, обобщённое ФСП, обезличенный опыт; имя, город, вуз и работодатели скрыты. Обоснование — MATCHING.md п. 8.1.
- **Пример (демо-сценарий).** Войдите через `/jury` → «Работодатель», откройте потребность «Backend Middle в платёжный шлюз» (`/employer/needs/:id`). У первого в подборке из 48 кандидатов `GET /api/needs/{id}/selection` отдаёт:
  - Score 69,3 = T 39,6 + K 19,7 + F 0 + A 0 + U 10,0 (сумма вкладов равна Score);
  - «за»: «Тест: верхние 10 % категории Backend (Python) · Middle, тест сегодня»; «Стек: подтверждено 3 из 3 обязательных — Python, FastAPI, PostgreSQL»; «Ожидания в вашей вилке ✓; гибрид подходит»;

- «против»: «Навык Kafka не подтверждён — проверьте на интервью» с ссылкой на навык брифа;
- `fsp_line`: «Достижения ФСП не привязаны — оценка по тесту и задачам». Первое место занято человеком без ФСП, и это нигде не в «против».

- **Пример (контрольные числа).** Три контрольных набора компонентов в тесте дают Score 83,2, 78,3 и 63,3. Второй (78,3) — без ФСП совсем (F = 0): он обгоняет третий, у которого сильное ФСП, но слабый тест (`test_control_order_strong_test_beats_strong_fsp`).
- **Доказательство.** `tests/test_matching_scoring.py` (контрольные примеры, детерминизм, `test_fake_declared_skills_shift_rank_by_zero`), `tests/test_matching_explain.py`, `tests/test_api_needs_privacy.py`. Валидация объяснений на 1000 карточках: Σ вкладов = Score — 100 %, довод ссылается на существующее доказательство — 100 %, без доказательства довода №1 Score падает — 100 % (VALIDATION.md п. 4.6). Качество выдачи — в разделе 7.

1.3. Приглашение с зарплатой и ответ «Да / Да, если… / Нет» — выполнено

- **Проблема.** Нужен сценарий выхода на контакт, где работодатель инициатор, но без спама и без игры в тёмную по деньгам.
- **Решение.** `POST /api/invitations`: описание, вилка в рублях, способ связи, срок ответа 3/5/7 дней; кандидат отвечает «Да», «Да, если…» (встречное условие: зарплата, формат, дата выхода) или «Нет». Кандидат видит приглашение с вилкой и компанией до ответа.
- **Пример (ошибки отправки).** Без вилки: `400 salary_required`, «Укажите вилку "от–до": без неё приглашение не отправляется». С вилкой 100 000–400 000: `409 contact_rejected`, «Вилка 100 000–400 000 ₽ шире 2,0× — сузьте её: кандидат должен видеть реальные условия» (мягкое предупреждение с 1,6×, блокировка с 2,0×: `config/contact.yaml → salary`). Там же второе правило: `card_not_opened` — пригласить можно только того, чью карточку вы открыли или закрепили в шортлисте.
- **Пример («Да, если…»).** В демо-мире у «Северного кода» есть приглашение со встречным условием: `counter.salary_from = 325000` при вилке 220 000–280 000, `summary`: «от 325 000 ₽ (ваша вилка 220 000–280 000 ₽)», `within_range: false`, `auto_disclose: true`, доступные действия `counter/accept`, `counter/decline`, `withdraw`, срок ответа на встречное — 3 дня.
- **Доказательство.** `tests/test_contact_invitation.py`, `tests/test_api_contact_lifecycle.py`, `tests/test_api_contact_deadlines.py`, `tests/test_contact_quotas.py`. Квоты (10 в пакете, 20 активных на бриф, 5 новых в неделю кандидату) подобраны симуляцией рынка, раздел 10.4.

1.4. Контакты открываются только после взаимного «да» — выполнено

- **Проблема.** Контакты и персональные данные нельзя отдавать до согласия человека; при этом после «да» стороны должны сразу получить возможность связаться.
- **Решение.** Два типа карточки: слепая (`CandidateBlind`) и раскрытая (`CandidateDisclosed`), правила видимости полей — `config/contact.yaml → visibility`. Раскрытие требует «да» кандидата и отдельного согласия (по умолчанию галочка стоит, снять можно; отозвать — в любой момент).
- **Пример.** `GET /api/needs/{id}/candidates/{ref}` для того же кандидата: до «да» `"view": "blind"`, после принятого приглашения `"view": "disclosed"`, и в ответе добавляется блок `disclosed` с полями `full_name`, `phone`, `email`, `telegram`, `city`, `current_employer`, `university`, `resume`, `fsp_extract`, `skills_passport`. В слепом ответе этих ключей нет ни на какой глубине.
- **Доказательство.** `tests/test_contact_privacy.py` (в слепом DTO нет ключей ПДн), `tests/test_api_contact_privacy.py`, `tests/test_api_needs_privacy.py`. Отзыв: `POST /api/invitations/{id}/revoke-consent`, чат закрывается (`test_chat_closes_when_consent_revoked_while_accepted`).

1.5. Фильтры и уточнения не теряют подборку — выполнено

- **Проблема.** Если работодатель добавил фильтр и «потерял» хороших кандидатов, доверие к системе падает.

- **Решение.** Подборка — неизменяемые версии. Фильтр не удаляет, а прячет: скрытые остаются в группе «Скрыто фильтрами» с причиной. К прежней версии можно вернуться (`POST /api/needs/{id}/rollback`), закрепить кандидата в шортлисте.
- **Пример.** `GET /api/needs/{id}/versions` отдаёт v1, v2…; ответ `selection` содержит `total_n` , `visible_n` , `hidden_n` , `hidden_summary` ; в таблице подборки (`/employer/needs/:id`) — блок «Скрыто фильтрами».
- **Доказательство.** `tests/test_matching_selection.py` , `tests/test_api_needs_flow.py` ; сборка версии: p95 133 мс на 1 865 кандидатов (VALIDATION.md п. 4.7).

1.6. Резюме и самоописание не влияют на категорию и ранг — выполнено

- **Проблема.** Если навыки можно написать в резюме, ранжирование можно «накрутить».
- **Решение.** Заявленные навыки — серые чипы «заявлено» и последний ключ сортировки при равном Score. В K входят только доказательства: тест, задачи, ФСП.
- **Пример.** В `/employer/needs/:id` у кандидата чип «Python» подписан источником («Тест: модуль Python ✓✓»), а Kafka — «нет данных». Заявленное самим кандидатом лежит отдельно (`declared_skills` , состояние `declared`) и рисуется серыми чипами; в карточке приглашения самоописание подписано «заявлено кандидатом».
- **Доказательство.** Накрутка: 50 слабым кандидатам каждого test-брифа дописали 5 навыков брифа, 1 000 случаев. Наш подбор сдвинулся вверх на 0,0 позиций (ни один не попал в топ-10), поиск по резюме (BM25) — на 518,4 позиции, 12,4 % попали в топ-10 (VALIDATION.md п. 4.4). Тесты: `test_fake_declared_skills_shift_rank_by_zero` , `test_self_declared_never_counts_in_k` .

1.7. Неподтверждённый грейд виден со статусом и опущен в выдаче — выполнено

- **Проблема.** Кандидат прошёл опрос, но не прошёл тест (или ещё не проходил). Скрывать такого человека нечестно, смешивать с подтверждёнными — тоже.
- **Решение.** Показываем с бейджем «грейд не подтверждён» и строкой «против» «Категория заявлена, тестом не подтверждена»; ранжируем ниже всех подтверждённых; в охват и перцентили он не входит.
- **Пример.** `POST /api/admin/import` с кандидатом `features: null` попадает в группу «категория по опросу, не подтверждена» и оказывается в хвосте; тот же бриф в `/lab/check` показывает группу отдельно (`n_unconfirmed` в ответе).
- **Доказательство.** `tests/test_api_unconfirmed_visibility.py` (`test_selection_ranks_unconfirmed_after_confirmed_and_explains`), `tests/test_matching_scoring.py::test_unconfirmed_survey_category_goes_below_confirmed` .

1.8. Поиск по банку без брифа — выполнено

- **Проблема.** Работодатель хочет просто посмотреть, кто есть на рынке, без описания потребности.
- **Решение.** `GET /api/bank` и экраны `/employer/bank` (карта талантов «специализация × грейд») и `/employer/bank/list` : специализация, грейд, категория, стек, сфера бизнеса, режим, «С ФСП». Сортировка `0,55·T + 0,25·F + 0,20·A` .
- **Пример.** В демо-мире фильтр «Backend (Python) · Middle» даёт 101 карточку; с включённым «С ФСП» (`has_fsp=true`) остаётся 17.
- **Доказательство.** `tests/test_api_bank_search.py` , `test_bank_sort_without_brief` ; MATCHING.md п. 11.

2. Категория и тест

2.1. Категория = специализация × грейд, определяется обязательным путём — выполнено

- **Проблема.** Нужна сопоставимая категория, которую нельзя получить словами.

- **Решение.** Опрос из четырёх шагов (сфера бизнеса, что делали руками, технологии и стаж, направление) → выбор грейда с брифингом → адаптивный тест → результат с категорией. Грейд присваивает только тест; стаж и ФСП на него не влияют. Справочник — `config/taxonomy.yaml`; с тестом четыре направления: Backend (Python), Frontend (JS/TS), QA, DevOps; остальные (Data/ML, Mobile, ИБ, Go, Java, UI/UX) — «по опросу», в подборке ниже подтверждённых.
- **Пример (демо-сценарий).** Войдите ступенчатым кандидатом `stage1-registered@…` и пройдите `/candidate/survey` → `/candidate/grade` → `/candidate/test/:id` → `/candidate/result/:id`; либо `stage4-category@…` — у него уже «Backend (Python) · Middle», балл 614, интервал 80 % — 535–694, уверенность 98 %, модули Python и SQL с ✓✓. Подсказка специализации по опросу: `POST /api/survey/hint`.
- **Доказательство.** Грейд только из теста: `tests/test_api_testing_rules.py`. Подсказка специализации: macro-F1 0,93 против 0,80 при чистом выборе кандидата, на отложенных профилях 0,92 (VALIDATION.md п. 3; `tests/test_survey_classifier.py`). Точность грейда: решение у заявленного порога верно в 0,86, точный грейд 0,748 (цель 0,75 — **не выполнено на 0,002**), с точностью ±1 — 1,00; грейд «по стажу» из опроса даёт 0,54 (VALIDATION.md п. 2.1).

2.2. Единый набор заданий не годится, «ИИ под каждого» ненадёжен — выполнено

- **Проблема.** Если у всех одно задание, оно утекает. Если генерировать задание нейросетью «под каждого», нельзя гарантировать ни правильность ключа, ни одинаковую сложность.
- **Решение.** Банк из 111 семейств заданий. Семейство — генератор: из одного шаблона получаются клоны с разными данными, ключ (правильный ответ) **вычисляется исполнением кода** (`run_python`, `run_js`, `run_sql`), а не пишется человеком или нейросетью. Каждый клон выдаётся одному человеку. Сложность семейства откалибрована экспертными оценками и ответами. LLM в тесте и ранжировании не участвует (проверяется тестом). Для Frontend есть JS-версии общих семейств: фронтендер не получает заданий на Python.
- **Пример.** `/lab/bank/:familyId` («3 клона рядом»): одно семейство, три разных экземпляра. Песочница жюри `/try` (раздел 2.3) показывает тот же механизм. Для жюри пересечение клонов у пар кандидатов — 0 из 67 974 выданных.
- **Доказательство.** Симуляция утечки: коалиция из 20 кандидатов сливает всё, что видела; ложный Middle у явных Junior — 0,10 против 1,00 у фиксированной формы и 0,77 у адаптивного теста без клонов (VALIDATION.md п. 2.6). Гейты банка — `tests/test_bank_gates.py` и `tests/test_bank_*.py` (банк целиком, включая JS-версии), `tests/test_frontend_no_python.py`, `tests/test_llm_boundaries.py`. Честная оговорка: клоны защищают от слива ответов, но не от слива идеи семейства; на «утечку метода» нужен CUSUM, и его задержка при 50 % знающих — 603 ответа при цели 200 (**не выполнено**, п. 2.7).

2.3. Защита от чат-ботов («Антиподсказчик») — выполнено, с оговоркой

- **Проблема.** Кандидат копирует условие в ИИ и вставляет ответ. Отдельный вопрос — как проверить такую защиту.
- **Решение.** Для каждого клона строится «двойник»: тот же текст с одним изменённым значением данных. Экран всегда показывает истинное условие, а в буфер обмена попадает двойник. Ответ, совпавший с ключом двойника, — улика для методиста. Оговорка: скриншот и ИИ-панель браузера двойник проходят; против них — «Подтверждение за 5 минут» (три вопроса по заданиям кандидата после «да»). Подробности — TESTING_MECHANISM.md п. 9.
- **Пример (живая проверка за минуту).** Откройте `/try`, скопируйте условие задания и вставьте его в любой ИИ или интерпретатор. В нашем прогоне на экране было `queue += [10]`, в буфере — `queue += [13]` (в вашей сессии значения будут другими). Решите версию из буфера и введите её ответ: итог «Поймано двойником: 2 из 4» (скопированы были два Python-задания), а акт сверки подсвечивает единственное изменённое значение (`10 → 13`) и показывает оба ключа рядом: `[72,39,24,7,10]…` по экрану и `[72,39,24,7,13]…` по копии. Те же шаги API: `POST /api/jury/cheat-lab` → `POST …/answers` → `GET …/result`. Песочница помечена `demo_assisted` и в метрики не идёт.

- **Доказательство.** `tests/test_anticheat_*.py` (240 тестов), e2e `frontend/e2e/tests/05-sandbox-twin.spec.ts`, пилот с ИИ-агентами — раздел 10.1. Двойник строится не для всех семейств: у 39 из 111 форма задания допускает безопасный двойник (ответ вводится текстом, язык исполняется офлайн); для остальных защита — клоны и сверка вживую (TESTING_MECHANISM.md п. 9.1).

2.4. Стабильность при повторе и честная смена грейда — выполнено (две цели валидации не достигнуты)

- **Проблема.** Один и тот же человек в разные дни не должен получать разные грейды; тест на ступень ниже не должен «сжигать» попытку; повышение не должно быть автоматическим прыжком.
- **Решение.** Гистерезис (ложное повышение 0,03), пересдача по правилам: повтор без грейда — через 14 дней, смена грейда — не чаще раза в 90 дней, не больше 3 попыток за 90 дней; провал на заявленном грейде открывает тест на ступень ниже сразу и без счётчика попыток; повышение — только отдельным тестом (`config/testing.yaml → rules`).
- **Пример.** Экран `/candidate/qualification` у `stage4-category@…`: «Сменить грейд можно не чаще раза в 90 дней: тест на повышение — через 89 дней» (`GET /api/tests/qualification`: `can_upgrade.code = "change_interval"`, `next_change_in = "через 88 дней"`). Для понижения: `POST /api/tests/qualification/{spec}/downgrade`.
- **Доказательство.** Совпадение решения при пересдаче 0,81 (цель 0,78, выполнено), к 0,551, QWK 0,86 — выполнено; **корреляция оценок 0,86 при цели 0,88 — не выполнено**, и мы объясняем почему: адаптивный тест ищет решение «выше/ниже порога», а не точную оценку каждого (VALIDATION.md п. 2.3, 6). Пилот с ИИ-агентами (раздел 10.1): повтор у 12 честных персон — точное совпадение полосы 0,83, различие не больше одной ступени 1,00. Тесты: `tests/test_api_testing_rules.py`, `tests/test_cat_*.py`. Свои профили: `make jury-check PROFILES=…` (раздел 7).

2.5. Короткий тест — выполнено

- **Проблема.** Длинный тест отпугивает кандидатов.
- **Решение.** Адаптивный тест останавливается, когда уверенность достаточна: от 8 до 20 заданий. Прогресс на экране — «от 8 до 20», а не фиксированный счётчик.
- **Пример.** Уверенный кандидат заканчивает на восьмом задании. В пилоте Middle-персоны ответили на 8 заданий, Junior — в среднем на 13,1 (от 8 до 20).
- **Доказательство.** В симуляции основная часть — в среднем 12,8 задания (цель ≤ 15), с доборами 16,1 (цель ≤ 17), p95 20 (VALIDATION.md п. 2.2).

2.6. Регулярные короткие задания от работодателей — выполнено

- **Проблема.** Профиль устаревает; работодателю нужен способ проверить конкретный подход.
- **Решение.** Тип A — задача из каталога откалиброванных семейств (свой клон, автопроверка), тип B — «предложи подход» с рубрикой из четырёх критериев по 0–2, решение уходит работодателю без имени. Лимиты: не больше 1 предложения в 7 дней кандидату, до 15 минут, отказ не штрафуется, после 3 пропусков — пауза 30 дней. Планировщик раз в час предлагает задания по каденции и напоминает за сутки до срока (`config/scheduler.yaml → task_offers`). Задачи влияют на свежесть профиля и подтверждение навыков, но не меняют грейд.
- **Пример.** `/employer/tasks` — создать задачу из каталога (`GET /api/tasks/catalog`); `/candidate/tasks` — предложение и таймер. Если компаний нет — «тренировочная задача от платформы».
- **Доказательство.** `tests/test_api_tasks_flow.py`, `tests/test_scheduler.py`; TESTING_MECHANISM.md п. 11.

2.7. Из описаний вакансий — задания и тесты — выполнено

- **Проблема.** Нужно показать, как по описанию вакансии строятся проверки, не ломая сопоставимость грейдов.

- **Решение.** Стандартный тест категории **под вакансию не меняется**. Вакансия даёт две вещи: «как мы проверим» (отображение навыков брифа на модули теста) и проверочный набор из 3–5 откалиброванных задач на конкретные навыки, суммарно до 15 минут, для кандидатов из шортлиста.
- **Пример.** `GET /api/needs/{id}/check` для брифа «Python, FastAPI, PostgreSQL, плюсом Kafka и Docker»: «Python ✓ проверено стандартным тестом категории»; «FastAPI ✓ проверяется проверочным набором по вакансии (модуль "FastAPI")»; «Kafka ✗ в стандартный тест категории не входит — проверьте на интервью». `POST /api/tasks/check-sets` собирает набор.
- **Доказательство.** 11 семейств заданий на PostgreSQL, FastAPI, Redis и очереди с ключом-исполнением; на 40 брифах набор поднимает P@10 с 0,66 до 0,71, прирост +0,048 [+0,017; +0,082] (MATCHING.md, раздел «Навыки, наблюдаемые через проверочный набор»; отдельный прогон `eval.checkset_eval`, перепроверен 9 октября); `tests/test_matching_skills.py`, `tests/test_bank_skills.py`.

2.8. Экспресс-тест: жюри проходит тест само за 3–5 минут — выполнено

- **Проблема.** Жюри должно пройти тест сам, но не тратить полчаса.
- **Решение.** `/jury` создаёт одноразового кандидата с готовым опросом; экспресс — до 8 заданий, около 4 минут, тем же движком и банком. Категорию экспресс не присваивает и честно пишет, когда точности не хватило. Режим жюри показывает правильный ответ (`GET /api/demo/jury/hint`); такие сессии помечены `demo_assisted` и в калибровку и метрики не попадают.
- **Пример.** Экспресс-тест с подсказками на Backend Middle закончился после 6 заданий результатом «Экспресс-оценка: Middle или выше · на 97 % уверены»; в ответе `confirmed_grade: null`, `category: null` (категорию даёт только полный тест), разбивка «Junior 3/3, Middle 2/2, Senior 1/1».
- **Доказательство.** `tests/test_cat_express.py`, `tests/test_api_demo_jury_entry.py`, e2e `01-jury-express.spec.ts`.

3. Кабинет кандидата

Требование	Решение и пример	Доказательство	Статус
Профиль вручную или из PDF-резюме с авторасознаванием (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы)	`/candidate/profile/edit` : загрузка PDF (`POST /api/candidates/me/resume`), черновик полей, кандидат подтверждает каждое (`/resume/confirm`); распознанное помечено «заявлено» и в Score не входит	`tests/test_resume_parser.py`, `tests/test_api_profile_resume.py`	выполнено
Стандартизированный PDF-профиль	`/candidate/profile/pdf` → `GET /api/candidates/me/profile.pdf` ; на странице QR со ссылкой `/v/{код}` — публичная сверка файла без входа	`tests/test_pdf_profile.py`	выполнено
Обязательный путь: опрос → грейд → тест → категория	раздел 2.1	`tests/test_api_survey_flow.py` , e2e `02-candidate-journey.spec.ts`	выполнено
Статус, история опросов и тестов, тест на ступень ниже или выше с ограничением частоты	`/candidate/qualification` , раздел 2.4	`tests/test_api_testing_rules.py`	выполнено
Входящие приглашения с вакансией и вилкой; принять, отклонить, «Да, если…»	`/candidate/invitations` , `/candidate/invitations/:id` ; в ответе `GET /api/invitations/inbox` приглашение содержит вилку «220 000–280 000 ₽ на руки», компанию, её надёжность, срок «через 5 дней»	`tests/test_api_contact_lifecycle.py`	выполнено
Список вакансий, самостоятельный отклик, статусы	`/candidate/vacancies` ; `POST /api/applications` ; отклик сразу раскрывает контакты по отдельному согласию	`tests/test_contact_application.py` , `tests/test_api_contact_vacancies.py`	выполнено

Требование	Решение и пример	Доказательство	Статус
Настройки: приватность, согласия на обработку и показ, привязка ФСП ID	`/candidate/privacy`, `/consents`, `/visibility`, `/fsp`; журнал согласий `GET /api/consents`; удаление аккаунта `DELETE /api/candidates/me`	`tests/test_account_erasure.py`, `tests/test_platform_access.py`	выполнено
Кандидат видит только свои приглашения и отклики	Роли и проверка владельца в каждом маршруте; матрица доступа по OpenAPI	`tests/test_security_matrix.py`	выполнено

4. Кабинет работодателя

Требование	Решение и пример	Доказательство	Статус
Профиль компании: описание, направление, контакты	`/employer/company`, `PUT /api/companies/me` (название, сфера, описание, способы связи, сайт, ИНН)	`tests/test_api_company_profile.py`	выполнено
Описание потребности: кто нужен, чем занимается команда, стек, специализация, грейд	`/employer/needs/new`: свободный текст плюс блок «о команде» (размер, роль, контекст стека, о команде); команда не участвует в разборе и Score, но попадает в приглашение	`tests/test_api_needs_features.py`, миграция `0014_need_team`	выполнено
Рекомендованные категории, конкретные кандидаты, обоснование попадания	`GET /api/needs/{id}/categories`: «Backend (Python) · Middle», вероятность 0,64, 47 кандидатов, охват вилки 220–280 тыс.: 32 из 47; панель доказательств	раздел 1.2	выполнено
Поиск по банку: специализация, грейд, категория, стек, ФСП	раздел 1.8	`tests/test_api_bank_search.py`	выполнено
Приглашение с описанием и вилкой, отслеживание статусов	`/employer/invitations` — воронка по статусам, у каждого карточка со сроками; приглашение не привязано к опубликованной вакансии	`tests/test_api_contact_lifecycle.py`	выполнено
Публикация вакансий, редактирование, входящие отклики	`/employer/vacancies`: «Создать вакансию» из потребности или сразу с формы (бриф создаётся автоматически); `POST /api/vacancies`, `PATCH`, `/close`; отклики с причиной отказа	`tests/test_api_contact_vacancies.py`	выполнено
Контакты кандидата скрыты до принятия или самостоятельного отклика	раздел 1.4	`tests/test_contact_privacy.py`	выполнено

5. ФСП ID, достижения, кандидат без истории

5.1. Связь профиля с ФСП ID и совместимость с Keycloak — выполнено на мок-реестре

- **Проблема.** Профиль должен привязываться к участнику ФСП, авторизация — быть совместимой с Keycloak.
- **Решение.** OIDC с PKCE, realm `fsp` в `deploy/keycloak/`, демо-реестр `services/fsp-mock/`; на экранах входа и регистрации есть кнопка «Войти через ФСП ID» (если настоящий ФСП ID недоступен, объясняет, что делать). Первый вход создаёт аккаунт и отправляет код на email.
- **Пример.** `/auth/login` → «Войти через ФСП ID»; либо `/candidate/fsp` → «демо-вход ФСП» (`POST /api/fsp/demo-login`, спортсмены — `GET /api/fsp/demo/athletes`).
- **Доказательство.** `tests/test_api_fsp_link.py`, `tests/test_fsp_provider.py`, `tests/test_fsp_mock.py`. Оговорка: реального реестра ФСП и их Keycloak мы не видели — работает мок и контракт, который мы предлагаем (FSP_INTEGRATION.md п. 4 и 9 — шаги перехода).

5.2. Кандидат без истории ФСП обрабатывается корректно — выполнено

- **Проблема.** Если ФСП даёт преимущество, кандидат без него оказывается в проигрыше — а это большинство людей.
- **Решение.** Без истории F = 0, «против» не попадает никогда; категория, тест и все остальные пути те же. Отдельно измеряем экспозицию.
- **Пример.** Первый в подборке «Backend Middle в платёжный шлюз» (раздел 1.2) — без ФСП, Score 69,3. В `/employer/needs/:id` над ним нет ни одной строки «против» про ФСП.
- **Доказательство.** `test_no_fsp_is_zero_not_penalty`, `test_no_history_gives_zero_f_and_neutral_line`; «Отсутствие ФСП никогда не “против”» — 100 % на 1000 карточек; условная экспозиция кандидатов без ФСП в топ-10 — 0,91, в топ-50 — 0,94 при допуске 0,80–1,25 (VALIDATION.md п. 4.5); доля подтверждённых тестом с ФСП 92,9 %, без ФСП 93,4 % (п. 2.10).

5.3. Как достижения ФСП двигают ранг, а не грейд — выполнено

- **Проблема.** Дисциплины и мероприятия несопоставимы, а явки можно накрутить.
- **Решение.** Вклад = результат (призёр/участник) × место в поле × число стартов (с потолком) × релевантность дисциплины специализации × свежесть (полураспад 24 месяца). Иерархии мероприятий нет: все дисциплины равноправны. Онлайн-явки без призовых мест считаются не больше двух раз. Роль в команде из заявки (`team_function`) показывается серым и в расчёт не входит. Грейд от ФСП не зависит.
- **Пример.** Первое место из 103 на соревновании по продуктовому программированию полгода назад, личный зачёт: VSS 75,4, для брифа Backend вклад в Score 9,0 балла. Двенадцать подряд онлайн-участий без призовых мест дают столько же, сколько два (FSP_INTEGRATION.md п. 5). На экране — «ФСП: Призёр всероссийских · алгоритмическое · 2025 · топ-3 · личный зачёт»; точное место до раскрытия не показывается.
- **Доказательство.** `tests/test_fsp_vss.py` (контрольные примеры, потолок участий, `test_hidden_result_is_excluded_everywhere`, `test_awaiting_protocol_result_is_not_counted`). Модель сверена с открытыми нормами федерации: МСМК в нормах нет и в расчёте отсутствует, сроки разрядов — КМС 3 года, I–III 2 года, МС бессрочно. Веса — экспертные допущения; нужна калибровка вместе с ФСП (FSP_INTEGRATION.md п. 10).

5.4. Схема интеграции с реестром ФСП — выполнено

- **Решение.** FSP_INTEGRATION.md: схема, предлагаемый контракт API реестра, формулы, согласия, несовершеннолетние, 10 шагов перехода на реальные системы. Контракт реализован мок-реестром `services/fsp-mock/` и клиентом `FSPProvider`.
- **Доказательство.** `tests/test_fsp_provider.py` проверяет клиента против мока, включая сбои реестра: при недоступном реестре берётся последний снимок, а сбой не выдаётся за «результатов нет» (`test_fallback_uses_snapshot_when_registry_down`, `test_fallback_outage_is_not_reported_as_no_results`).

6. Зарплата, правила видимости, безопасность

Требование	Решение и пример	Доказательство	Статус
Вилка в рублях обязательна в вакансии и приглашении, подписана «на руки» или «до вычета НДФЛ»	`config/contact.yaml → salary`: шаг 1 000, минимум 30 000, предупреждение с 1,6×, блокировка с 2,0×; в API `salary_type_label: "на руки"`; примеры ошибок — раздел 1.3	`tests/test_contact_invitation.py`, `tests/test_contact_quotas.py`	выполнено
Приглашение содержит предложение, вилку, компанию и способ связи	`GET /api/invitations/inbox`: `salary.text` «220 000–280 000 ₽ на руки», `company` с надёжностью и меткой проверки	`tests/test_api_contact_lifecycle.py`	выполнено
Статусы приглашений и откликов видны обеим сторонам	У работодателя колонки воронки (отправлено, просмотрено, встречное условие, принято…), у кандидата `status_label`, сроки в `timers`	`tests/test_api_contact_deadlines.py`	выполнено

Требование	Решение и пример	Доказательство	Статус
Регистрация по email с подтверждением	`/auth/register`, `/verify-email`; письма в Mailpit `:8025`. В режиме жюри подтверждение отключено для удобства (`VZ_DEMO_AUTO_VERIFY_EMAIL=false` возвращает код)	`tests/test_platform_auth.py`	выполнено; в демо упрощено
Верификация компании и модерация вакансий не обязательны в MVP	Всё равно есть добровольная метка «Проверенная компания» и жалобы: раздел 10.9	`tests/test_trust_verification.py`	выполнено
152-ФЗ: отдельные согласия на обработку и публикацию, журнал, удаление	`/candidate/consents`; `GET /api/consents/documents` — тексты; `DELETE /api/candidates/me`	`tests/test_account_erasure.py`, SECURITY.md	выполнено
Роли, защита от несанкционированного доступа, пароли и токены	argon2id, короткий access и refresh в `HttpOnly`-cookie с ротацией, ограничение частоты; матрица «каждый защищённый маршрут × чужая роль» по схеме OpenAPI	`tests/test_platform_security.py`, `tests/test_security_matrix.py`	выполнено
Кандидаты 15–17 лет с согласием представителя	`config/contact.yaml → minors`: моложе 15 регистрация отклоняется; 15–17 — профиль виден работодателям только после согласия законного представителя (письмо со ссылкой на 7 дней), приглашать можно только на стажировку	`tests/test_minors_guardian.py`	выполнено
LLM открыто и вне оценки	LLM.md: разбор брифа и резюме с цитатой, по умолчанию выключен; ранжирование, тест и ключи — без LLM	`tests/test_llm_boundaries.py` — 0 обращений к модели в тесте и подборе	выполнено

Про слепую карточку: организаторы допускают показ имени и прежних мест работы, мы **сознательно не показываем** их до «да» (меньше предвзятости; основания — MATCHING.md п. 8.1). Вместо них — обезличенный опыт («Backend, 2 г. 3 мес.»).

7. Валидация: собственная и на ваших данных

- **Проблема.** Нужна своя процедура валидации и доказательство, что механика работает; жюри хочет проверить на закрытом наборе.
- **Решение.** Протокол записан до прогона (VALIDATION_PLAN.md: метрики, пороги, бейзлайны, правило решения; хэши файлов брифов), результаты — VALIDATION.md. Синтетическая популяция из 2 000 кандидатов с известным скрытым уровнем, 100 брифов (20 dev, 20 test, 60 предрегистрированных test2 с заморозкой SHA-256), оракул релевантности по скрытой истине, бейзлайны: поиск по резюме (BM25), категория + точечный уровень, случайный порядок, фиксированная форма и «ИИ под каждого» для теста. `make validate` — около 6 минут.
- **Пример (результат, который можно сверить).** На 20 test-брифах P@10: наш сквозной подбор 0,61 [0,49; 0,73], поиск по резюме 0,13 [0,07; 0,19], случайный порядок в категории 0,25. На 60 новых брифах, правило решения записано заранее: 0,54 против 0,36 у «категория + уровень» (разность +0,18, ДИ [0,12; 0,24], значима) и 0,14 у поиска по резюме. Сводка — 38 из 43 пороговых пунктов (из исходных 41 — 36).
- **Пример (на ваших данных, без Swagger).** Войдите через `/jury` → «Лаборатория» → `/lab/check`. Вкладка «Наш набор пар» одной кнопкой даёт метрики нашего набора (20 брифов, 1 706 пар): P@10 0,615 [0,485; 0,725], поиск по резюме 0,13, категория + уровень 0,49; по каждому брифу — топ-10 с компонентами Score и «за/против» для ручной проверки. «Своя вакансия против банка платформы» — вставляете вакансию с вилкой и видите подборку. «Свой набор» — загружаете три таблицы (вакансии, кандидаты, разметка `rel` 0–3) в CSV или JSON; шаблоны скачиваются на странице; методист видит только свои загрузки. Для теста: `make jury-check PROFILES=backend/eval/jury_profiles_example.jsonl` прогоняет ваши профили через боевой движок (на примере из 16 профилей при `--seed 1`: повтор точно 0,81, ±1 1,00, QWK 0,92, r 0,92).

- **Доказательство.** `tests/test_eval_*.py`, `tests/test_api_admin_check.py`, `tests/test_api_admin_access.py`, `tests/test_jury_simple.py`. Протокол пилота — `backend/eval/agent_pilot/PROTOCOL.md`.
- **Честно, что не выполнено (5 из 43).**
  1. Точный грейд 0,748 при цели 0,75 — в пределах выборочной ошибки (на пяти других сидах в среднем 0,758).
  2. Корреляция оценок при пересдаче 0,86 при цели 0,88 — несовместима с бюджетом длины теста.
  3. Задержка детектора утечки CUSUM при 50 % знающих — 603 ответа при цели 200; он остаётся сигналом методисту.
  4. P@10 значимо выше «категория + уровень» на 20 test-брифах — не значимо (разность 0,12, ДИ [-0,04; 0,28]); на всех 40 брифах значимо, на 60 новых — значимо, но абсолютный P@10 0,54 ниже порога 0,60.
  5. Стабильность топ-10 при пересдаче 0,28 при цели 0,70 — цель поставлена некорректно, это показано расчётом; постфактум-метрики выполнены.
- **Статус.** Частично: процедура и числа на синтетике воспроизводимы, но экспертной разметки пар и живого пилота на людях нет. Оракул и генератор писала одна команда, тексты брифов — с ИИ-ассистентом.

8. Документы, API, запуск, нагрузка

Требование	Где и как проверить	Статус
Архитектура: функциональная и компонентная	ARCHITECTURE.md: C4, ERD, sequence-диаграммы четырёх сценариев, машины состояний	выполнено
Механика тестирования, механика подбора и категоризации, валидация	TESTING_MECHANISM.md, MATCHING.md, VALIDATION_PLAN.md, VALIDATION.md	выполнено
Схема интеграции с ФСП / ФСП ID	FSP_INTEGRATION.md	выполнено
API: запросы, ответы, коды ошибок	API.md; Swagger `/api/docs`, ReDoc `/api/redoc`, `/api/openapi.json`. У каждой из 242 операций есть описание и коды 4xx; ошибки в едином формате `{"code", "message", "details"}`, например `salary_required`, `contact_rejected`. Проверка: `tests/test_platform_openapi.py`	выполнено
Библиотеки с версиями	THIRD_PARTY.md из `uv.lock` и `package-lock.json`, лицензии открытые	выполнено
Пошаговый запуск, Docker	DEPLOY.md, `docker-compose.yml`, три Dockerfile; на чистой машине `docker compose up -d --build` и `make demo`. Пул заданий догенерируется около 2,5 минуты: до его конца тест недоступен (в `GET /api/health` поле `pool_ready`). Стенд с HTTPS на VPS одной командой: `sudo bash deploy/bootstrap.sh`	выполнено; `bootstrap.sh` на настоящем VPS не запускали
Производительность при десятках пользователей	LOAD_TEST.md: 100 одновременных пользователей, 3 минуты, 24 739 запросов, 0 сбоев, p50 31 мс, p95 170 мс, p99 300 мс; найдены и исправлены deadlock и N+1 (воронка приглашений: p95 4,9 с → 150 мс). Нагрузка с отдельной машины не проверялась	выполнено
Код открыт, без обфускации, лицензия	`LICENSE` (MIT); стек на открытых лицензиях; единственное исключение — `axe-core` (MPL-2.0) как dev-зависимость e2e, в образы не попадает	выполнено
Развёрнутый прототип по ссылке	**Не развёрнут**: нужен VPS; скрипт готов (`deploy/bootstrap.sh`). Работает локально за 5 минут	не сделано

9. Ответы организаторов: что учтено

Пересказ ответов организаторов (встреча и таблица вопросов) и то, как они отражены. Номера в последнем столбце — разделы этого документа.

Что ответили организаторы	Что сделано	Где
Грейд — только по тесту категории, не по вакансии и не по ФСП; стаж — только атрибут	`Qualification` собирается из теста; ФСП влияет на ранг	2.1, 5.3
Неподтверждённый грейд показывать со статусом и опускать ниже	Хвост выдачи с бейджем	1.7
Кандидат сам выбирает грейд, подсказка допустима	`/candidate/grade`, `POST /api/survey/hint`	2.1
Повышение — отдельным тестом; провал не сжигает попытку; кулдаун на смену	90 дней на смену, 14 на повтор, тест на ступень ниже сразу	2.4
Одинаковые задачи не должны расходиться между кандидатами; защита от ИИ и списывания — большой плюс; песочница и прокторинг	Клоны, «Антиподсказчик», события страницы теста (копирование, вставка), `/try`	2.2, 2.3
LLM допустимы, облачные тоже, использование описать	LLM.md: GigaChat проверен вживую на 62 брифах, прироста нет — честно записано; ПДн маскируются; по умолчанию выключен	6, 10.12
Датасет пар собирает команда, жюри проверит руками; своя процедура, минимума по объёму нет	100 брифов, `/lab/check`, `make jury-check`, импорт	7
Реальный Keycloak не ждут, вход через имитацию ФСП ID — ожидаемый сценарий, регистрация по email тоже	Мок-реестр, realm `fsp`, кнопка на экране входа	5.1
Контакты открываются пригласившему после принятия; отзыв доступа и чат на платформе — плюс	Раскрытие после «да», `revoke-consent`, чат после взаимного «да»	1.4, 10.7
ФСП: дисциплины равны, считать число стартов и результат, не набивать онлайн-явками; срок давности можно ввести	Потолок участий, класс результата, свежесть с полураспадом 24 месяца	5.3
Задачи прошлых соревнований ФСП как дополнительный буст	Не реализовано (дали бы преимущество решавшим)	11
Несколько категорий — плюс; хватит двух специализаций; дизайнеры — приоритет у технических ролей	Несколько квалификаций у одного кандидата, Fullstack как производная; UI/UX — «по опросу»	2.1
Зарплата: подписать гросс/нетто	«на руки» / «до вычета НДФЛ» везде	1.3, 6
Несовершеннолетние 15–17 с согласием родителей — плюс; нужен правовой минимум (152-ФЗ)	Согласие представителя, согласия и журнал	6
Экспресс-тест 3–5 минут; жюри пройдёт тесты само	`/jury`, до 8 заданий	2.8
Проверка компании — добровольная метка	«Проверенная компания» не блокирует сценарий	10.9
Регулярные задания адресно; эталон проверяет система, открытый ответ — работодатель	Тип A (автопроверка) и тип B (рубрика)	2.6
Задания под вакансию — отдельный блок; единый тест на категорию, но не с одинаковым пулом	Проверочный набор, клоны	2.7
ATS: синхронизация вакансий, откликов, статусов; в MVP не нужна, концепция идёт в плюс	Pull-API, вебхуки, адаптер «Хантфлоу» на эмуляторе	10.8
Парсинг допустим только для тестовых данных	Открытые данные рынка без ПДн для словаря и вилок	10.6
Качество подбора важнее скорости; GPU и ограничений по образам нет	p95 сборки версии 133 мс, без локальных моделей	1.5, 8
Питч 10 минут; видео вместо живого стенда допустимо, стенд держать на время оценки	Презентация и видео собираются отдельно; стенд не развёрнут	8, 11
Исходный код открыт, идею после дедлайна не менять, смотрят даты коммитов	Лицензия MIT, история коммитов не переписывалась	8

10. Сверх кейса

Возможности, которых кейс не требовал, но которые делают механику честнее или проверяемее. Для каждой — пример и доказательство; ограничения указаны.

10.1. Пилот с ИИ-агентами: проверка «Антиподсказчика» и повтора теста

- **Что.** Мы не ограничились симуляцией: Claude-агенты играли кандидатов с заданным профилем знаний и проходили настоящий тест через настоящий HTTP-API (регистрация, опрос, выбор грейда, клоны, ответы, результат). 12 честных персон (4 специализации × Junior/Middle/Senior) и 2 «списывающие» (знают на Junior, заявляют Middle и решают каждое задание копией из буфера), по 2 попытки. Протокол и метрики записаны до прогона (`backend/eval/agent_pilot/PROTOCOL.md`).
- **Как устроена защита сейчас.** Двойник ставится на каждое задание сессии, где он посчитан (`twin_rate_per_session: 20`); предрасчёт двойников входит в конвейер пула (сервис `pool` , `make gen-pool`), так что на свежем стенде защита работает с первой сессии; выдача отдаёт сначала клон с готовым двойником.
- **Результат для «списывающих».** В 4 прогонах: статус «не подтверждён», ложного Middle нет; двойник сработал на 4–7 заданиях из 6–8 скопированных. Тест выдачи: `tests/test_api_testing_pool.py::test_issue_prefers_instance_with_ready_twin` . Эти 4 прогона сделаны после доработки выдачи и в предрегистрированные доли не входят; предыдущая версия и её результат записаны в VALIDATION.md п. 10.4 как фактическая запись.
- **Остальные результаты пилота** (VALIDATION.md раздел 10): повтор у честных — точное совпадение полосы 0,83, ±1 — 1,00, взвешенная к 0,83, корреляция баллов 0,97; все Middle-персоны подтвердили заявленный Middle (8 из 8), но по баллу попали в полосу Senior — ИИ «не умеет не знать», это предсказано в протоколе.
- **Оговорки.** Выборка маленькая (12 + 2 персоны), это ИИ, а не люди; **одна ложная тревога на 24 честных прогона при ориентире 0**; скриншот и ИИ-панель браузера двойник не ловит — для них «Подтверждение за 5 минут».

10.2. Цифровой паспорт навыков (W3C VC, QR)

- **Что.** Результат теста и достижение ФСП кандидат выпускает как подписанный файл (Verifiable Credential 2.0 и Open Badges 3.0, подпись Ed25519, издатель `did:web`) — переносится между платформами; проверка по QR и по файлу, отзыв.
- **Пример.** `/candidate/profile/passport` → выпустить «Призёр · продуктовое программирование (хакатон) · 2025» → скачать файл → изменить в нём один символ → загрузить на `/v/` : проверка вместо «Паспорт подлинный: подписан издателем, не изменён, не отозван, срок не истёк» отвечает «Проверка не пройдена: Подпись не совпадает: содержимое файла изменено после выдачи» (шаг `signature` : `ok: false`). То же по API: `POST /api/credentials/me` , `GET …/file` , `POST /api/credentials/verify` .
- **Доказательство.** `tests/test_credentials_core.py` , `tests/test_credentials_api.py::test_tamper_one_byte_turns_red` , `test_revoke_flow_and_status_list` ; CREDENTIALS.md. Издатель демонстрационный: «документ не является официальным сертификатом ФСП»; путь к настоящему издателю описан в документе.

10.3. Панель ФСП «Кадровый резерв»

- **Что.** Для федерации: воронка от соревнований до найма — привязан ФСП ID → прошёл тест → подтверждена категория → попал в подборки → получил приглашение → «Взаимно» → нанят; в разрезе дисциплин и округов, спрос и предложение, медианы оферт. Только агрегаты, k-анонимность не меньше 5 со вторичным подавлением.
- **Пример.** `/jury` → «Панель ФСП» (`/fsp/reserve`). На демо-мире: 45 привязанных против 353 без ФСП; доля дошедших до подборки (от прошедших тест) 90,7 % против 52,8 %, до «Взаимно» (от получивших приглашение) 59,5 % против 33,3 %. Ячейки меньше 5 показываются как «<5». Над цифрами предупреждение: «Демо-данные синтетические … Числа показывают, как панель работает, а не состояние реального кадрового резерва».
- **Доказательство.** `tests/test_fsp_dashboard.py` , `config/fsp_dashboard.yaml` ; выгрузка `GET /api/fsp/dashboard/export.csv` ; FSP_INTEGRATION.md п. 11.

10.4. «Сигнал интереса» и симуляция рынка

- **Что.** Вторая сторона обратной механики: кандидат отмечает компанию (2 сигнала за 30 дней, живёт 60, отзыв слот не возвращает, отдельное согласие). Работодатель видит бейдж «Взаимный интерес» только у карточек, которые сам открыл или закрепил; в Score сигнал не входит. Числа квот выбраны прогоном синтетического рынка.
- **Пример.** `/candidate/vacancies/:id` → «Отправить сигнал интереса (осталось 2 из 2)»; у работодателя в банке `interest=true` оставляет только отметивших, а `interest_first=true` ставит их первыми — Score и порядок остальных не меняются.
- **Доказательство.** `tests/test_interest_signal.py` (`test_employer_sees_signal_only_after_card_opened`, `test_bank_badge_filter_sort_and_score_unchanged`), `tests/test_market_sim.py`. Симуляция (MATCHING.md п. 9.1): квоты меняют заполнение вакансий на нагрузку — 10 приглашений на вакансию вместо 44 (нагрузка на кандидатов ниже в 4,4 раза) ценой 2 п.п. заполнения (88,6 % против 90,6 %); сигнал почти не меняет число совпадений (+0,5 ± 0,5 п.п.), но повышает выигрыш кандидатов (0,79 против 0,64). **Граница, которую мы показываем сами:** при похожих вкусах всех кандидатов сигнал пользы не даёт. Это модель с допущениями, а не измерение живого рынка.

10.5. Мониторинг качества заданий

- **Что.** DIF по Мантелю–Хензелю (справедливость задания для групп ФСП/регион/режим доступности/экспресс), монитор дрейфа сложности, контроль экспозиции семейств. Экран `/lab/quality`.
- **Пример.** Метод проверен на внедрённых смещённых заданиях: из четырёх найдены все (классы B, C); среди 316 пар «настоящее семейство × группа» класс C — 0, класс A — 98 %. На утечке 12 семейств монитор нашёл 12 из 12 без ложных флагов среди остальных.
- **Доказательство.** `tests/test_quality_monitoring.py`, VALIDATION.md раздел 8. Это проверка метода на синтетике; выводов о реальных группах людей нет. Контроль экспозиции Симпсона–Хеттера в продукте выключен: за 8 итераций он сходится не полностью, это написано.

10.6. Открытые данные рынка

- **Что.** Словарь навыков и справочная рыночная вилка из открытых данных портала «Работа в России» (Trudvsem), без ПДн. Вилка — подсказка в форме брифа, в Score не входит.
- **Пример.** `GET /api/needs/market-salary?specialization=backend_python&grade=middle` → `n: 11`, `net.p50: 87000`, источник и дата выгрузки; в поле `note` предупреждение: «Выборка по всей РФ, региональные вакансии занижают уровень относительно Москвы и Санкт-Петербурга». Выборка маленькая (n = 11) — и API это показывает.
- **Доказательство.** `tests/test_market_salary.py`, `config/market.yaml`; MATCHING.md п. 13.

10.7. Чат после взаимного «да»

- **Что.** Переписка открывается только после раскрытия контактов и закрывается при отзыве согласия; сообщения ограничены по длине и частоте.
- **Пример.** После принятия приглашения кандидат видит экран «Взаимно» с контактом компании и пишет в чат; `GET /api/chat/threads`.
- **Доказательство.** `tests/test_api_chat.py` (`test_chat_closed_before_mutual_yes`, `test_chat_closes_when_consent_revoked_while_accepted`), e2e `04-mutual-chat.spec.ts`.

10.8. ATS и эмулятор «Хантфлоу»

- **Что.** Pull-API `/api/ats/v1` по ключу компании, исходящие вебхуки с подписью HMAC-SHA256 и повторами, обратная синхронизация статусов, адаптер «Хантфлоу» с эмулятором.
- **Пример.** В профиле компании укажите адрес `…/api/demo/webhook-sink` — события приглашений видны на `GET /api/demo/webhook-sink` сразу, с проверкой подписи; журнал адаптера показывает запросы ровно в

том виде, в каком они ушли бы в Хантфлоу.

- **Доказательство и оговорка.** `tests/test_api_ats.py`. **Против живого Хантфлоу не запускалось**, повторов боевых запросов к ATS нет (ATS_INTEGRATION.md п. 1).

10.9. Доверие и безопасность платформы

- **Что.** Метка «Проверенная компания» (добровольная, никогда не блокирует сценарий): контрольные суммы ИНН и ОГРН, поиск в учебном реестре, совпадение названия и домена. Жалобы кандидатов на приглашения и вакансии (анонимны для компании). Три жалобы от разных кандидатов за 90 дней снижают квоту приглашений компании вдвое; подтверждённая жалоба на фиктивную вакансию, просьбу о деньгах или сбор резюме снимает метку. Автосигналы для модератора («Много приглашений, ни одного принятого»), которые ничего не блокируют.
- **Пример.** `/employer/company` → ввести ИНН и ОГРН → уровень «Реквизиты проверены»; в приглашении кандидат видит «Не проверена» или метку. Жалобы разбирает методист в `/lab/trust`.
- **Доказательство и оговорка.** `tests/test_trust_verification.py`, `tests/test_trust_complaints.py`, `config/trust.yaml`. Реестр учебный (вымышленные компании), настоящий ФНС не подключён.

10.10. Сравнение кандидатов бок о бок

- **Что.** В подборке работодателя флажок «к сравнению» (до 3), плавающая панель и окно сравнения: только слепая карточка, Score с разложением вкладов, свежесть теста, «нет истории ФСП — не штраф», подсветка лучшего значения, приглашение прямо из колонки.
- **Пример.** `/employer/needs/:id` → отметить трёх кандидатов → «Сравнить». Файлы: `frontend/src/features/employer/compare.ts`, `components/selection/CompareDialog.tsx`.
- **Доказательство и оговорка.** Типы и сборка фронтенда в CI; автотеста интерфейса для этого окна нет.

10.11. Страница `/lab/check`, «Обещание процесса», `bootstrap.sh`

- `/lab/check` — проверка подбора на ваших данных без Swagger: раздел 7.
- **«Обещание процесса».** Работодатель может отметить в приглашении, что кандидата ждёт «Без технического скрининга — результат теста засчитан», «Сразу финальное интервью» или «Без тестового задания»; кандидат видит обещания до ответа. После «да» в сверке «условия совпали?» он может отметить «Обещание не выполнено» — это событие снижает надёжность компании (при низкой надёжности квота приглашений уменьшается вдвое); в Score кандидата не входит. Тест: `tests/test_contact_invitation.py::test_outcome_check_promise_broken_is_a_reliability_minus`, API-тест полного цикла — `tests/test_api_contact_lifecycle.py::test_process_promises_flow_and_promise_broken_check`; неизвестный код — `400 promise_unknown`. Строки — `config/contact.yaml → invitation.promises`.
- `deploy/bootstrap.sh` — стенд одной командой на чистом сервере (Docker, swap, секреты, HTTPS через Caddy на `<ip>.sslip.io`, пул, двойники, демо-мир, проверка health). Проверены синтаксис и `docker compose config`; на настоящем VPS не запускался.

10.12. Прозрачное использование LLM

- **Что.** Шлюз `app/llm/` с режимами `none/replay/record/live`, детерминированным fallback и маскированием ПДн. В контуре оценки (ключи, грейд, Score, ранжирование, вердикты «Антиподсказчика») LLM запрещена архитектурно.
- **Пример.** GigaChat подключён вживую (TLS по корневому сертификату Минцифры), 62 синтетических брифа: F1 навыков 0,975 и «только правила», и «правила + LLM» — **прироста нет**, и документ так и говорит. Кассета позволяет воспроизвести прогон без ключа.
- **Доказательство.** `tests/test_llm_gigachat.py`, `tests/test_llm_boundaries.py`, LLM.md п. 9.

10.13. Инженерия, доступность, безопасность, нагрузка

- **CI** (`.github/workflows/ci.yml`): линт `ruff` , `mypy --strict` на ядрах, `pytest` на PostgreSQL с pgvector тремя шардами, проверка миграций вверх и вниз, фронтенд (`tsc` , `eslint` , сборка, `npm audit`), e2e Playwright, сборка образов, `pip-audit` . Оговорка: после последних слияний мы разбираем таймауты на двухъядерном раннере GitHub; зелёный прогон на текущей ветке до сдачи не подтверждён.
- **Тесты.** Более 2 400 тестов бэкенда (собрано 2 424 командой `pytest --collect-only`), 6 сценариев e2e; контрольные примеры из спецификаций — обязательные тесты.
- **Доступность.** Автоматическая проверка axe-core (WCAG 2.1 A/AA) на 13 экранах: было 115 нарушений контраста и ссылок, стало 0 serious и critical; кабинет работодателя без горизонтального скролла на 390, 768 и 1440 px (ACCESSIBILITY.md). Автоматика ловит около трети проблем, ручная проверка выборочная.
- **Безопасность.** Матрица «защищённый маршрут × чужая роль» по схеме OpenAPI (`tests/test_security_matrix.py`), CSP, argon2id, ротация refresh-токенов, `pip-audit` и `npm audit` без уязвимостей в прод-зависимостях (ENGINEERING.md). Динамический скан (OWASP ZAP) не запускался.
- **Нагрузка.** 100 пользователей: 24 739 запросов, 0 сбоев, p95 170 мс (LOAD_TEST.md).

11. Чего нет и что частично

Список, который мы бы хотели знать сами, будучи жюри.

Что	Статус	Почему и что дальше
Развёрнутый стенд по ссылке	не сделано	Нужен VPS; скрипт `deploy/bootstrap.sh` готов, но не запускался на настоящем сервере. Локально работает за 5 минут
Проверка на живых людях	не сделано	Вся валидация — синтетика и ИИ-агенты; экспертной разметки пар нет. Оракул и генератор написала одна команда
5 из 43 порогов валидации	не выполнено	Перечислены в разделе 7 с причинами; пороги после прогона не менялись
Реальные ФСП и ATS	эмуляция	Мок-реестр, мок ЕГРЮЛ, эмулятор «Хантфлоу». Контракты и шаги перехода описаны
Защита от скриншота и ИИ-панели браузера	частично	Двойник ловит только копирование текста; против остального — «Подтверждение за 5 минут» (живая сверка), а любой удалённый тест от ИИ-ассистента не защищён полностью. Грейд — решение с уровнем доверия, а не сертификат
Покрытие двойниками	частично	39 из 111 семейств допускают безопасный двойник (ответ текстом, исполняемый язык); для остальных — клоны и сверка
Задачи прошлых соревнований ФСП как буст	не сделано	Дало бы преимущество решавшим
Автоматические тесты интерфейса	частично	6 e2e-сценариев; окно сравнения кандидатов и ряд экранов покрыты только типами и сборкой
CI зелёный на GitHub	сделано	Все 9 заданий зелёные на `main` ; известный обход: завершение процесса pytest сразу после отчёта (сбой pydantic_core при выходе интерпретатора, WORKLOG п. 8)
Подтверждение email в режиме жюри	упрощено	В `DEMO_MODE` код не требуется; в боевом режиме включён
Внутренние ограничения	см. документы	Лимиты запросов хранятся в памяти процесса (при нескольких воркерах мягче), общий rate limit — в плане (ARCHITECTURE.md п. 9)